

Acceleration of FFT-512 on CV32A6 via CV-X-IF Coprocessor Team LumiEirb

6th National RISC-V Student Contest 2025–2026

Sponsored by Thales, GDR SOC2 and CNFM

Fatima Zahra El Meady Meryem Elouassai Leila Founti

RISC-V Contest Team — Zybo Z7-20, Vivado 2023.2, Ubuntu 22.04

May 2026

Abstract. We present a hardware/software co-design approach to accelerate FFT-512 on the CV32A6 RISC-V processor. A custom coprocessor, connected via the CV-X-IF interface, implements fixed-point complex arithmetic including a fused radix-2 butterfly instruction. The KISS FFT library is refactored from recursive to iterative, specialized for $N = 512$. Together, these optimizations reduce the FFT cycle count from **167 569** (reference) to **98 951 cycles** on the Zybo Z7-20 FPGA board, a **41% cycle reduction**. All benchmarks (FFT, CoreMark, MNIST) validate correctly on hardware, confirming full RV32IM_Zicsr compatibility.

1 Introduction and Motivation

The Fast Fourier Transform (FFT) is one of the most widely deployed algorithms in digital signal processing, communications, and scientific computing. Despite its $O(N \log N)$ complexity, computing FFT-512 on a scalar in-order RISC-V processor still involves thousands of fixed-point complex arithmetic operations, making it a natural candidate for hardware acceleration.

The 6th National RISC-V Student Contest tasks participants with reducing the number of clock cycles required by a **CV32A6**-based system-on-chip to compute an FFT of 512 complex samples. Competitors may modify the CV32A6 microarchitecture or attach a coprocessor, provided all correctness constraints are respected and the operating frequency does not decrease by more than 20%.

Our approach combines two complementary strategies:

1. **Hardware acceleration:** a custom coprocessor connected via **CV-X-IF** implementing six complex arithmetic instructions, including a fused radix-2 butterfly instruction.
2. **Software optimization:** complete refactoring of the KISS FFT library from a generic recursive implementation to an iterative, size-specialized routine that exploits the hardware instructions.

The starting point is the reference project hosted at <https://github.com/ThalesGroup/cva6-softcore-contest>, branch `cv32a6_contest_25_26`. The reference FFT executes in 167 569 cycles on the Zybo board and 153 496 cycles in the reference setup.

2 System Architecture

2.1 CV32A6 and the CV-X-IF Interface

The CV32A6 is a 32-bit, 6-stage in-order RISC-V processor implementing the RV32IMACZicsr instruction set. It integrates an HPDCache data cache, a CVA6 I-cache, and supports the **CV-X-IF** coprocessor interface.

CV-X-IF defines three handshaking channels between the processor pipeline and an external execution unit:

- **Issue channel:** the processor forwards an instruction opcode and up to two register values to the coprocessor.
- **Result channel:** the coprocessor returns the computation result and a destination register index.
- **Commit channel:** the processor signals whether the instruction commits or is flushed.

This architecture cleanly decouples the custom execution unit from the main pipeline, allowing the coprocessor to be developed and verified independently.

2.2 Data Representation

All computations use fixed-point arithmetic to avoid floating-point hardware and the associated area overhead. Complex numbers are packed into 32-bit registers using the format:

$$\text{reg}[31 : 0] = \underbrace{\{\text{imag}[15 : 0]\}}_{\text{upper 16 bits}}, \underbrace{\{\text{real}[15 : 0]\}}_{\text{lower 16 bits}}$$

Values are stored in **Q15 format**: a signed 16-bit integer where the value v represents the real number $v/2^{15}$. Multiplications produce Q30 intermediate results (32-bit signed), which are rounded back to Q15 using:

$$x_{Q15} = \frac{x_{Q30} + 2^{14}}{2^{15}}$$

This rounding halves the quantization error compared to truncation and preserves the signal-to-noise ratio of the FFT output within acceptable bounds for a per-bit comparison with the reference.

3 Coprocessor Design

3.1 Custom Instruction Set

Six custom instructions are defined within the RISC-V **custom-3** opcode space, ensuring forward compatibility with the standard ISA. Table 1 summarizes the instructions.

Table 1: Custom instructions implemented in the CV-X-IF coprocessor

Instruction	Operation
CMUL_I16	$\text{rd} = x_r \cdot y_r - x_i \cdot y_i + j(x_r \cdot y_i + x_i \cdot y_r)$
CADD_I16	$\text{rd} = \text{rs1} + \text{rs2}$ (complex Q15 add)
CSUB_I16	$\text{rd} = \text{rs1} - \text{rs2}$ (complex Q15 sub)
SETTW_I16	$\text{shadow_tw} \leftarrow \text{rs1}$ (load twiddle factor)
BFLY2_I16	$y_0 = \text{rs1} + t$; $\text{shadow_y1} = \text{rs1} - t$, $t = \text{rs2} \cdot \text{shadow_tw}$
GETY1_I16	$\text{rd} \leftarrow \text{shadow_y1}$

3.2 Fused Butterfly Instruction

The butterfly is the fundamental computation of the Cooley-Tukey FFT algorithm. A radix-2 butterfly with twiddle factor tw takes two complex inputs (x_0, x_1) and produces two outputs:

$$y_0 = x_0 + x_1 \cdot tw, \quad y_1 = x_0 - x_1 \cdot tw$$

A naive implementation requires: one complex multiplication (4 real multiplications, 2 additions), then two complex additions. Using scalar RISC-V instructions, this amounts to approximately 8–12 instructions per butterfly. The BFLY2_I16 instruction computes both y_0 and y_1 in a single hardware cycle:

1. The twiddle factor tw is preloaded into an internal shadow register via **SETTW_I16**.
2. **BFLY2_I16 rd, rs1, rs2** computes $t = \text{rs2} \cdot \text{shadow_tw}$ (using 4 DSP48E1 multiplications), then $y_0 = \text{rs1} + t$ (written to rd) and $y_1 = \text{rs1} - t$ (stored in shadow_y1).
3. **GETY1_I16 rd** retrieves y_1 from shadow_y1 into the register file.

Due to the CV-X-IF constraint of a single register writeback per instruction cycle, y_1 cannot be written in the same instruction as y_0 . The shadow register approach satisfies this constraint while keeping the total butterfly cost at only **3 custom instructions** (vs. ~ 10 standard instructions).

3.3 Hardware Implementation

The coprocessor ALU (**copro_alu**) leverages Xilinx **DSP48E1** blocks to efficiently implement fixed-point complex operations.

A total of **10 DSP48E1 blocks** are used within the coprocessor on the XC7Z020 FPGA:

- 4 DSP48E1 blocks are used to implement the parallel real multiplications required for complex multiplication in **CMUL_I16** and **BFLY2_I16**

- 4 DSP48E1 blocks are used in the arithmetic datapath for accumulation and rounding of Q30 intermediate results back to Q15
- 2 DSP48E1 blocks are used for additional internal datapath operations

At the top-level design, a total of **14 DSP48E1 blocks** are reported, including the DSP resources already present in the baseline CV32A6 processor.

The coprocessor wrapper (`cvxif_example_coprocessor`) introduces a limited hardware overhead, using 326 LUTs and 105 flip-flops. The decode logic maps instruction encodings to the corresponding arithmetic operations and routes operands to the appropriate execution units within the coprocessor.

4 Software Optimization: KISS FFT Refactoring

4.1 Limitations of the Original Implementation

The original KISS FFT is a highly generic C library designed to support arbitrary FFT sizes through dynamic factorization. For a fixed 512-point FFT, this generality incurs substantial overhead:

- **Recursive execution:** the function `kf_work` calls itself recursively for each FFT stage, causing function call overhead, stack frame allocation, and register spills.
- **Dynamic factorization:** the FFT size is factored at runtime to determine the radix decomposition, adding unnecessary control logic.
- **Generic macros:** `C_MUL`, `C_ADD`, `C_SUB` expand to multi-instruction floating-point or fixed-point sequences that cannot be efficiently pipelined.
- **Branch prediction pressure:** nested conditionals in the generic butterfly functions cause frequent mis-predictions on the CV32A6 branch predictor.

4.2 Iterative Specialized Implementation

The refactored FFT is a **fully iterative implementation** for $N = 512 = 2^9$.

Unlike the original generic KISS FFT, the recursion and dynamic factorization have been completely removed. The computation is explicitly structured as a sequence of nested loops, directly mapping the FFT stages to memory access patterns optimized for the hardware coprocessor.

This implementation is manually tailored for the 512-point case, allowing better control of data locality, reduced branching, and efficient integration of the custom butterfly instructions.

Twiddle factors $tw_k = e^{-j2\pi k/N}$ are precomputed offline in Q15 format and stored in a lookup table of 256 entries, corresponding to the unique coefficients required for the 512-point FFT.

4.3 Impact on Instruction Count

For a 512-point FFT, the total number of butterflies is $\frac{N}{2} \log_2 N = 256 \times 9 = 2304$. With the fused instruction, each butterfly requires 3 custom instructions plus approximately 4 load/store instructions for memory access, yielding roughly $2304 \times 7 \approx 16,000$ operations for the butterfly core. The remainder of the 73 182 measured instructions comes from loop overhead, twiddle factor table lookups, and function preamble/postamble.

5 FPGA Implementation Results

5.1 Experimental Setup

- **Target board:** Digilent Zybo Z7-20 (Xilinx XC7Z020CLG400-1)
- **EDA tool:** Vivado 2023.2 (synthesis + place & route)
- **Host:** Ubuntu 22.04.4 LTS, 64-bit
- **Programming:** GDB + OpenOCD over JTAG; serial output via `picocom` at 115200 baud
- **Note:** Due to limited access to a QuestaSim license during the project, full RTL simulation logs could not be generated locally. QuestaSim was initially used within the university environment to analyze the baseline CV32A6 core, but access became constrained during the development phase due to internship periods and restricted availability of licensed tools. Functional validation was therefore performed directly on the Zybo Z7-20 FPGA board, including FFT, CoreMark, and MNIST benchmarks. The provided repository remains fully reproducible and can be executed in a complete simulation environment if required.

5.2 Performance Results

Table 2: FFT-512 Performance: Reference vs. Optimized Design

Metric	Reference	Optimized	Change (%)
Instructions (board)	124 142	73 182	−41.1%
Cycles (board)	167 569	98 951	−41%

The FFT output passes the per-bit correctness check in all board runs, confirming that fixed-point rounding does not introduce numerical drift.

5.3 Timing Analysis

The post-place-and-route timing report reveals a critical path violation of -2.180 ns on the setup path from the scoreboard commit pointer to the coprocessor ALU result register. The path traverses 30 logic levels including 2 DSP48E1 blocks, 10 CARRY4 chains, and multiple LUT stages.

Table 3: Timing Summary after Place & Route

Parameter	Reference	Optimized
Target clock period	25.000 ns	25.000 ns
Worst setup slack	+0.066 ns	-2.180 ns
Data path delay (critical)	≈ 24.9 ns	27.026 ns
Logic levels (critical)	–	30
$f_{\max}$	≈ 40.1 MHz	≈ 36.8 MHz
Frequency change	–	−8.1%

The maximum frequency is computed as $f_{\max} = 1/(t_{\text{period}} - \text{slack})$:

$$f_{\max, \text{ref}} = \frac{1}{(25.000 - 0.066) \times 10^{-9}} \approx 40.1 \text{ MHz}$$

$$f_{\max, \text{opt}} = \frac{1}{(25.000 - (-2.180)) \times 10^{-9}} \approx 36.8 \text{ MHz}$$

The 8.1% frequency reduction is well within the 20% limit imposed by the contest rules. The design operates correctly at the board frequency, which remains below the estimated maximum frequency.

5.4 FPGA Resource Utilization

Table 4: FPGA Resource Utilization — Top-Level Design (XC7Z020)

Resource	Reference	Optimized	Available	Change
Total LUTs	19 782	20 714	53 200	+4.7%
Logic LUTs	19 542	20 474	53 200	+4.8%
LUTRAMs	200	200	–	0%
Flip-Flops	16 438	14 775	106 400	−10.1%
RAMB36	60	60	140	0%
RAMB18	5	5	280	0%
DSP Blocks	4	14	220	+10

The addition of the coprocessor and the software refactoring introduces a limited resource overhead compared to the reference design. The total LUT count increases slightly, while the number of flip-flops remains nearly unchanged. This overhead is mainly due to the integration of the CV-X-IF coprocessor datapath.

The use of **14 DSP48E1 blocks** (6.4% utilization) reflects the cost of accelerating complex arithmetic operations in hardware. This additional DSP usage remains limited with respect to the available resources and is justified by the significant performance gain achieved.

6 Functional Validation on FPGA

6.1 FFT Benchmark

The FFT was loaded onto the Zybo board via GDB/OpenOCD and executed with UART output captured through picocom:

Listing 1: FFT UART output on Zybo Z7-20 board

```
1 FFT running...
2 FFT finished
3 kiss_fft took 73182 instructions and 98951 cycles
4 SUCCESS : fft result values correct
```

The SUCCESS message confirms a per-bit match with the golden reference, validating both the correctness of the hardware butterfly arithmetic and the software loop structure.

6.2 CoreMark Benchmark

CoreMark was executed *without recompilation* to verify RV32IM_Zicsr compatibility:

Listing 2: CoreMark UART output

```
1 2K performance run parameters for coremark.
2 CoreMark Size      : 666
3 Total ticks        : 1190293
4 Total time (secs): 1.190293
5 Iterations/Sec     : 2.520388
6 Iterations         : 3
7 Correct operation validated.
8 CoreMark 1.0 : 2.520388 / GCC13.1.0 / BRAM
```

CoreMark completes successfully. The score (2.52) is not a criterion for this year’s contest but demonstrates that the coprocessor extensions are transparent to standard RV32IM code.

6.3 MNIST Benchmark

MNIST digit recognition was executed without recompilation:

Listing 3: MNIST UART output on Zybo Z7-20 board

```
1 Expected = 4
2 Predicted = 4
3 Result : 1/1
4 credence: 82
5 image env0003: 1760387 instructions
6 image env0003: 3514034 cycles
```

The network correctly predicts digit **4** with credence **82**, exactly matching the reference specification. This confirms that the custom opcode extensions do not interfere with unmodified binaries and that the base processor pipeline remains fully functional.

7 Discussion

7.1 Analysis of the Performance Gain

The 41% cycle reduction results from two interacting effects:

Instruction count reduction (−41.1%): Each of the $256 \times 9 = 2304$ butterflies now requires 3 custom instructions instead of approximately 10 standard instructions, reducing the butterfly instruction count by $\sim 70\%$. Combined with the elimination of recursive function calls and dynamic factorization, the total instruction count drops from 124 142 to 73 182.

Cycle-per-instruction improvement: The iterative structure reduces control overhead and eliminates recursive function calls, leading to fewer pipeline disruptions. In addition, the custom instructions execute in a single cycle and produce results immediately, reducing data hazards and minimizing pipeline stalls compared to multi-instruction scalar implementations.

7.2 Critical Path and Timing Limitation

The timing violation (-2.180 ns) is concentrated on the path from the scoreboard commit pointer through the coprocessor pipeline. The 30-level combinational depth results from the interaction between the issue-channel decode logic, the DSP multiply-accumulate chain (including 2 DSP48E1 blocks in series), and the rounding carry chain (10 CARRY4 cascades).

A straightforward mitigation would be to insert a **pipeline register** between the second DSP48E1 output and the rounding CARRY4 chain. This would:

1. Break the critical path into two segments of approximately 13–14 ns each.
2. Allow the coprocessor to close timing above 70 MHz.
3. Introduce a 1-cycle latency increase per butterfly, adding at most 2304 extra cycles to the FFT — a negligible 2.3% increase.

This optimization was not completed within the contest timeframe but represents the immediate next step.

7.3 Contest Rule Compliance

- **Coprocessor-only approach:** our optimization is entirely based on the CV-X-IF coprocessor extension. No modification was made to the CV32A6 pipeline, memory subsystem, caches, or AXI interconnect.
- **Custom opcode space:** all custom instructions use the RISC-V **custom-3** opcode space, ensuring compliance with the unprivileged specification.
- **Memory transparency:** all data accesses (FFT buffer, twiddle table) go exclusively through the CV32A6 load/store unit. No DMA, scratchpad, or memory-mapped acceleration was introduced.
- **Frequency constraint:** maximum frequency decreases by only 8% ($\approx 40.1\text{ MHz} \rightarrow \approx 36.8\text{ MHz}$), well within the 20% limit.
- **Non-regression:** CoreMark and MNIST binaries are not recompiled and pass correctly on hardware.
- **Functional correctness:** FFT returns SUCCESS on the Zybo board with per-bit accuracy against the reference.

8 Conclusion

We have demonstrated that hardware/software co-design on the CV32A6 RISC-V processor can significantly accelerate FFT-512 computation. By combining a CV-X-IF coprocessor implementing a fused radix-2 butterfly instruction with an iterative, size-specialized FFT implementation, we have achieved:

- **41% cycle reduction** on the Zybo Z7-20 board (98 951 vs. 167 569 cycles)
- **41.1% instruction count reduction** (73 182 vs. 124 142 instructions)
- **4.7% LUT increase** due to coprocessor integration
- **10.1% FF reduction**, mainly due to software simplification
- **8.1% maximum frequency decrease**, remaining within the 20% constraint
- Successful execution of FFT, CoreMark, and MNIST benchmarks on hardware without recompilation

The CV-X-IF interface enables a clean integration of custom hardware while preserving compatibility with the base processor. The fused butterfly approach can be extended to higher radices (e.g., radix-4) to further improve performance.

This work highlights the effectiveness of fine-grain acceleration using custom instructions within the CV-X-IF framework.

Repository: <https://github.com/fizera-gi/LumiEirb>

Report: <https://github.com/fizera-gi/LumiEirb/tree/main/cva6-softcore-contest/report>

Branch: cv32a6_contest_25_26

Submission date: May 3rd, 2026